

Versionshantering

Session 2 · Branches, merge-konflikter, pull requests, code review

Mjukvaruutvecklingsprocessen — DevOps

Arcada · 8.9–20.10.2026

Läget: resultat från M1?

- Repot finns, skapat från template
- Commit med taggen `m1-repo` är uppe på GitHub

Fastnade du? Läraren hjälper under labben.

Agenda idag

1. Läget efter M1
2. **Föreläsning:** branches, merge, konflikter, PR, code review
3. Paus
4. **Lab M2** — branch protection, buddy-review, en riktig konflikt
5. Wrap-up: verifiera M2, vanliga problem, nästa gång

Del 1: Branches och merge

Varför branches?

Ert repo är snart en **produkt i drift**: i M9 deployar varje merge till `main` automatiskt till produktion.

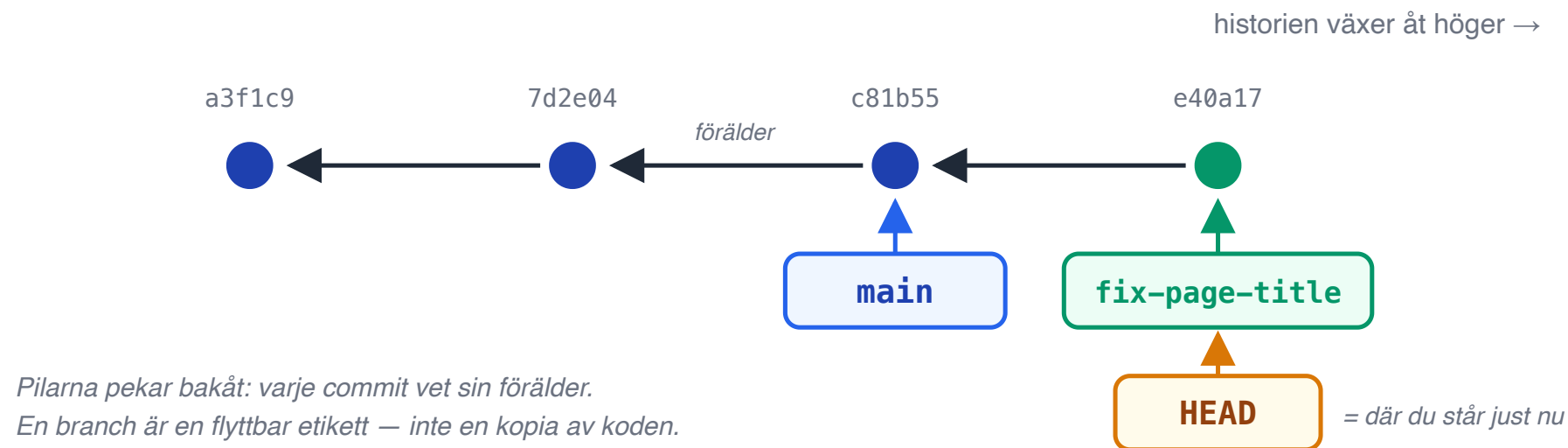
- `main` måste därför **alltid fungera** — den är inte er lekplats
- En **branch** är ett säkert arbetsutrymme: experimentera fritt, `main` påverkas inte förrän ni väljer att merge
- Namnge efter **vad** den gör: `add-get-item`, `add-color-theme` — eller med typ-prefix: `fix/broken-title`, `feature/delete-button` — inte `test`, `ny`, `asdf2`

```
git switch -c add-get-item    # skapa och hoppa till ny branch
git switch main              # hoppa tillbaka
git branch                   # lista brancher, * = där du står
```

Vad är en branch egentligen?

En branch är bara en **flyttbar pekare till en commit** — inte en kopia av koden.

- Att skapa en branch är **gratis** (en 41-bytes fil i `.git/`) — skapa många, radera ofta
- Varje commit pekar på sin **förälder** → historien är en graf
- **HEAD** = pekaren till **där du står just nu**



Merge: förena två historier

```
git switch main  
git merge add-get-item
```

Två fall:

1. **Fast-forward** — `main` har inte rört sig sedan branchen skapades → Git flyttar bara pekaren. Ingen ny commit.
2. **Merge-commit** — båda har nya commits → Git skapar en commit med **två föräldrar** som förenar dem.

GitHubs *Merge pull request*-knapp gör fall 2 (den skapar alltid en merge-commit).

Merge-konflikt: så här ser den ut

```
<<<<<<< HEAD
    <h1>Bobs anteckningar</h1>
=====
    <h1>Alices anteckningar</h1>
>>>>>>> origin/main
```

- **Ingen krasch, ingen förlorad kod** — Git ställer en fråga den inte kan besvara: *båda sidor ändrade samma rad — vilken gäller?*
- `<<<<<<< HEAD` → **din sida** (branchen du står på)
- Under `=====` → **deras sida** (det du mergar in, här: `origin/main`)
- Resten av filen är redan förenad — bara de markerade raderna är frågan

Konfliktlösning i fyra steg

1. **Öppna filen** — sök efter `<<<<<<`
2. **Bestäm**: din version, deras, eller en **kombination** (vanligast!)
3. **Radera markörerna** — filen ska se ut som du vill att resultatet ska se ut
4. Tala om för Git att du är klar:

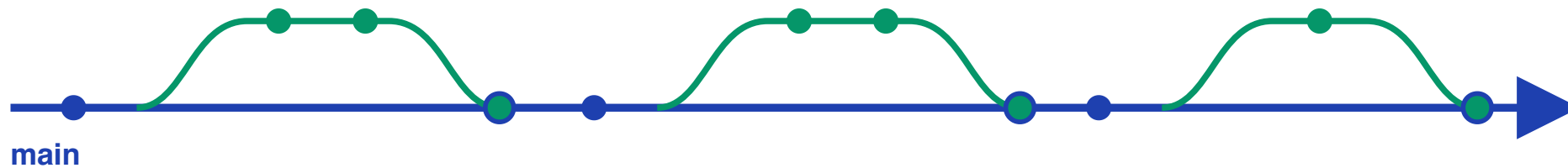
```
git add frontend/index.html  
git commit                # färdigt förslag på meddelande finns redan
```

Panik? `git merge --abort` ångrar hela mergen — tillbaka till läget före. Andas, försök igen.

Därför: små branches, täta merges

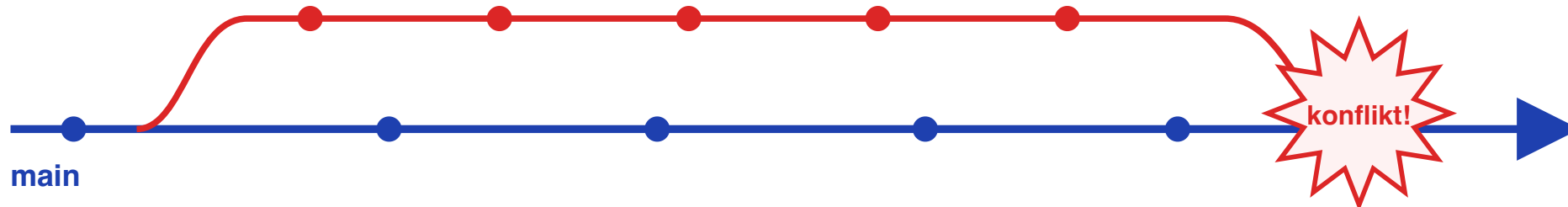
Korta branches, täta merges

små ändringar → små, lätta merges



Långlivad branch, sen merge

två versioner av verkligheten glider isär



Konfliktens storlek växer med **tiden** två historier glider isär — inte med er skicklighet.

Del 2: Pull requests och code review

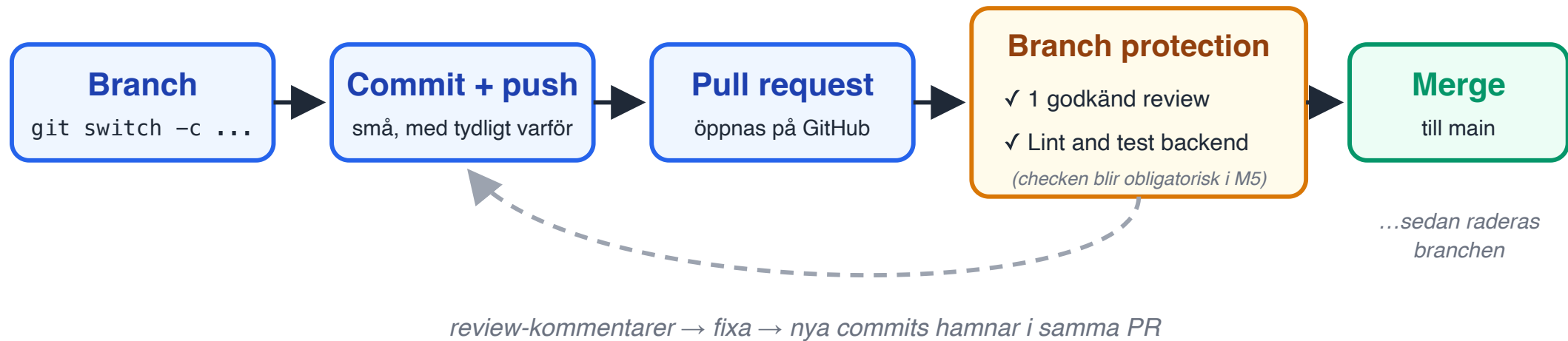
Pull request: ett förslag + ett samtal

En PR är **inte** en Git-funktion — den är GitHubs lager ovanpå: *"jag föreslår att den här branchen mergas till main — vad säger ni?"*

- **Diffen** — exakt vad som ändras, rad för rad
- **Samtalet** — kommentarer, frågor, förslag — *innan* koden landar
- **Checkarna** — CI kör automatiskt på varje PR (kolla Actions-fliken: `Lint and test backend` har kört på ert repo sedan M1)
- Pushar du fler commits till branchen hamnar de i **samma PR** — så åtgärdar man review-kommentarer

Flödet ni kör från och med idag

Från och med M2: ingen annan väg in i main



Review: samma kommentar, två sätt

✗ Så här inte	✓ Så här
"Det här är fel."	" <code>fetch</code> saknar felhantering — om backend är nere visas en tom sida. Kan vi visa ett felmeddelande?"
"Varför gjorde du så??"	"Jag hade väntat mig <code>switch</code> här — vad fick dig att välja <code>if/else</code> ? Kanske missar jag något."
"Snyggt!! LGTM 🚀" (utan att ha läst)	"Kollat: körde appen på 8080, temat syns och knappen är läsbar. En fundering på rad 12, annars klart."

Mönstret: peka på **rad**, säg **varför**, föreslå **väg framåt** — och ställ frågor i stället för att döma.

No hello: hela frågan direkt

✗ Så här inte	✓ Så här
"Hej, har du tid?" (<i>väntar på svar</i>)	"Hej — git-fråga: kör jag X får jag Y , vad missar jag?"
PR-kommentar: "Kolla min PR?"	PR-beskrivning: vad, varför, var — allt granskaren behöver för att agera

Skicka inte en hälsning och vänta — lägg hela frågan, med kontext, i **första** meddelandet. Gäller PR-beskrivningar och Slack/Teams-chattar lika mycket som review-kommentarer. (Se nohello.net.)

Reviewn ska läsas — särskilt när AI skrev koden

Claude Code, Codex och Gemini ligger i devcontainern. Codex har en gratisnivå (per 9.9.2026) — den som har en egen prenumeration på Codex, Gemini eller Claude kan använda den i stället.

- **Som reviewer:** läs diffen tills du förstår den — en LGTM på kod ingen läst är dubbelt tom när diffen kom ur en prompt
- **Som författare:** läs din egen diff *innan* du öppnar PR:en. Kan du inte förklara varje rad är den inte klar för review
- Läs **kommentarerna du får tillbaka** — svar på riktiga frågor, inte formaliteter
- **I labben:** uppdrag A/B får göras med AI — men PR:en står alltid i **ditt** namn

AI genererar — vi granskar som människor. Ansvaret för `main` är ert.

Reviewer-checklista för M2

När din buddy ber om review — **eller när du granskar din egen PR (solo)** — kontrollera och **säg vad du kontrollerat**:

- ☐ **Förstår jag ändringen** — oavsett vem eller vad som skrev den? Om inte — fråga i en kommentar (det är en review-insats, inte ett misslyckande)
- ☐ **Diffen innehåller bara det PR:en handlar om** — inga överraskningsfiler (`.DS_Store` , `node_modules` , hemligheter)
- ☐ **Commit-meddelandena** säger *varför*, inte bara *vad*
- ☐ **Checken är grön** — om `Lint and test backend` är röd: titta i loggen tillsammans
- ☐ **Inga kvarglömda konfliktmarkörer** (`<<<<<<`) i diffen

Godkänn med en mening om vad du kollat — solo: **Comment**, inte Approve.

Det sociala kontraktet

- Kommentera **koden, inte personen** — *"den här funktionen"*, aldrig *"du"*
- **Författaren ≠ koden**. En hittad bugg är en vinst för teamet — någon fann den före produktionen
- Reviewerns fråga är en **gåva**: den betyder *"jag lägger min tid på att förstå ditt arbete"*
- **Godkänn inget du inte läst** — ett ärligt "jag hinner inte förrän eftermiddagen" slår en falsk LGTM

Lab: M2

Branch protection · buddy-review · en riktig konflikt

Lab M2 steg 3 — två uppdrag

Uppdrag A — backend	Uppdrag B — frontend
Ändring: <code>GET /api/items/{item_id}</code> — itemet, eller 404 <code>Item not found</code> . Plus två nya tester : träff + 404.	Ändring: färgtema som CSS-variabler i <code>style.css</code> — <code>--bg</code> / <code>--fg</code> / <code>--accent</code> på <code>body</code> och <code>button</code> , <code>#ddd</code> blir variabel.
Reviewern kollar: checkar ut branchen, <code>cd backend && pytest</code> grön — båda fallen täckta.	Reviewern kollar: <code>docker compose up --build -d</code> , port 8080 — läsbar text, synlig knapp, inga färgvärden kvar.

Solo: välj **ett**. Så *testar du* i PR-beskrivningen

M2 — dagens milstolpe

Mål: repot är skyddat, och er första riktiga PR är mergad efter review.

1. Slå på en **ruleset** på `main`: kräv PR — **Required approvals 1 (par) / 0 (solo)**
2. Branch → **Uppdrag A eller B** (se labben) → PR med avsnittet *Så testar du*
3. **Review:** checklista, kommentera, fixa, godkänn, **merge** — radera branchen
4. **Konfliktövningen:** båda ändrar samma rad enligt instruktionen — den andra PR:en får en äkta konflikt att lösa
5. Tagga: `git tag m2-review` — och pusha taggen

Solo: ingen approve på egen PR — egen radkommentar (*Comment*) i stället, sedan merge.

Bevis: mergad PR (solo: självgranskad) + löst konflikt + taggen uppe + branch protection i `inlamning/m2-review.md`.

Wrap-up: är M2 klar?

- [] Branch protection är **på**: en push direkt till `main` avvisas
- [] Minst en **mergad PR per person** — par: med godkänd review av buddyn · solo: med dokumenterad självgranskning i PR:en
- [] Konflikten är **löst och mergad** — inga `<<<<<<`-rester (`git grep '<<<<<<'` är tyst)
- [] Gamla brancher **raderade** på GitHub
- [] Taggen `m2-review` syns under **Tags**, och det **osynliga arbetet** är dokumenterat i `inlamning/m2-review.md`

Fastnade du? Ta det **olöst** till session 3 — M1–M9 bör vara klara till session 10.

Tagg satt för tidigt? Flytta den

Committade ni mer **efter** `git tag m2-review` — t.ex. bevisfilen kom in sent? Då pekar taggen på fel commit. **Flytta taggen till committen där milstolpen faktiskt blev klar** — inga force-flaggor, radera och återskapa:

```
git tag -d m2-review  
git push origin --delete m2-review  
git tag m2-review          # på rätt commit  
git push origin m2-review
```

Det här är en **korrigering**, inte en vana — sikta på att tagga när ni faktiskt är klara.

Nästa gång

Session 3 · ti 15.9 kl 13:00 — Containers

Images, layers, registries — och ni öppnar appens riktiga Dockerfiles (i repot sedan M1).

M3: bygg frontend + backend som images, kör helheten med `docker compose`, pusha till GHCR.

Ta med: ett M2-klart repo — och från och med nu: **allt arbete via PR.**

Ni har nu kursens arbetssätt. Resten av kursen ändrar bara *vad* som går genom det — aldrig *hur*.